

SQL - Structured Query Language

- ⊛ Databases କଠିକି ଏକତ୍ର କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।
- ⊛ Case Sensitive ଟାଇପର ଲେଖାଯାଇଥାଏ।

SELECT statement

- ⊛ database ଥିବା data select କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।
- ⊛ ଯେଉଁ ଯେଉଁ columns select କରିବା ପାଇଁ କିଛି columns ଥିବାର କାରଣ ଦେଖିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।

ଉଦା: SELECT CustomerName, City
FROM Customers;

* ଯଦି 'Customers' ନାମର demo database ଥିବାର CustomerName ଓ City columns ଥିବାର select କରାଯାଏ।

ଉଦା: SELECT * FROM Customers;

* ଯଦି କିଛି columns ବାଛି select କରାଯାଏ।

SELECT DISTINCT statement

- ⊛ Different values ଥିବାର return କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।

ଉଦା: SELECT DISTINCT Country FROM Customers;

* ଯଦି 'Customers' database ଥିବାର Country column ଥିବାର ଭିନ୍ନ ଭିନ୍ନ values return କରାଯାଏ।

- ⊛ Different values ଥିବାର କାରଣ ଦେଖିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।

SELECT COUNT (DISTINCT Country) FROM Customers;

- ⊛ ଯଦି କିଛି କାରଣ ଦେଖିବା ପାଇଁ MS Access database ବ୍ୟବହୃତ ହୁଏ।

* Chrome, Safari, Opera browsers ଉପରେ ଯଦି ଯେଉଁ SQL statement ଥିବାର run କରାଯାଏ।

* ଯଦି statement ଥିବାର Edge browser ଉପରେ run କରାଯାଏ।

SELECT Count (*) AS Distinct Countries
FROM (SELECT DISTINCT Country FROM Customers);

WHERE Clause

★ WHERE condition select records from database table.
આવાસીઓ તે સ્પષ્ટ.

★ SELECT statements માટે જોઈએ છે; UPDATE, DELETE statements માટે નહીં.
તે સ્પષ્ટ છે.

ઉદા: 1) SELECT * FROM Customers
WHERE Country = 'Mexico';

2) SELECT * FROM Customers
WHERE CustomerID = 1;

★ WHERE clause માં જે =, >, <, >=, <=, <> (not equal) * એવા operators
હોય છે.

* <> - not equal. In some versions of SQL this operator
may be written as !=

★ જેવાં કે જેવાં operators હોય છે.

1) BETWEEN

SELECT * FROM Customers
WHERE Price BETWEEN 50 AND 60;

2) LIKE (Search for a pattern)
SELECT * FROM Customers
WHERE City LIKE 's%';

3) IN

SELECT * FROM Customers
WHERE City IN ('Paris', 'London');

AND, OR and NOT operators

★ WHERE clause માં AND, OR, NOT operators હોય છે.

ઉદા: 1) SELECT * FROM Customers
WHERE Country = 'Germany' AND City = 'Berlin';

2) SELECT * FROM Customers
WHERE Country = 'Germany' OR City = 'Berlin';

3) SELECT * FROM Customers
WHERE ~~Country~~ NOT Country = 'Germany';

★ AND, OR and NOT operators combined શકે છે, જોઈએ છે.

- 1

* ଚେରାଣି ମୁଗାରେ ଫିଟେଇତ
ତହାଜାଉଛି.

⑧ ପ୍ରକାଶନାଂଶୁ ହିରୋ (Descending order) ଉପାୟାନୁସାରେ,

(Ascending order)

- ```
SQL:- SELECT* FROM Customers
 ORDER BY Country, CustomerName;
```

④ જોડણી Columns થી Ascending, Descending orders માં માં આપવામાં આવે છે.

```
2201- SELECT * FROM Customers
 ORDER BY Country ASC, CustomerName DESC;
```

### INSERT INTO statement

- ④ ଚେନ ମୁଦାର ଗୁଣାକାର ଛୋଟ ଧାନ୍ତି ମୁଦର; ଚେନ ଛୁଇଁବେଳେ ଗ୍ରାସ ଲେଉଟା ଗୁଣା.

- ```
INSERT INTO Customers (CustomerName, ContactName, City)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Stavanger');
```

- INSERT INTO Customers

VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', ...)

- * Columns କିଲେବାର data insert କରା ଯିବ; ଡ୍ରାପ୍ ଅଫ୍ ବାଉନ ୧, ଡ୍ରାପ୍ ନାହିଁ

- * ଡାଟାମି କ୍ରମରେ ୧ column ଗଠନକାରୀ ଅଟେ; table ଥିବା column ମୂଳା ଡାଟାମି
values ଡାଟାମି ଡାଟାମି.

NULL Values

- ⑤ NULL values ໃນ fields ອາດ; values ບາດເຈນ fields ເລີຍ.

- Ⓐ values add କେବଳ କ୍ଷେତ୍ରରେ ଥିବା NULL Value field ଥିବା ବିଚାରକୁ ଧ୍ୟାନ ଦିଅନ୍ତୁ।

- ④ NULL values ဆိုတာ testing වဲင် 'comparison operators' ဆိုက် တော့တာ.
 လို ဆိုရာ IS NULL ဆို IS NOT NULL operators ဆိုက်.

૨૨૦- SELECT CustomerName, ContactName, Address
FROM Customers
WHERE Address IS NULL;

૨૨૦- SELECT CustomerName, ContactName, Address
FROM Customers
WHERE Address IS NOT NULL;

UPDATE Statement

⊛ table માંથી records modify કરી શકાય છે.

૨૨૦- CustomerID = 1 એ Customer માટે ContactName અને City update કરો.

```
UPDATE Customers  
SET ContactName = 'Alfred', City = 'Frankfurt'  
WHERE CustomerID = 1;
```

૨૨૦- Country = 'Mexico' એ માટે records એ ContactName update કરો.

```
UPDATE Customers  
SET ContactName = 'Juan'  
WHERE Country = 'Mexico';
```

Warning!

★ UPDATE કરી શકાય છે WHERE clause નો ઉપયોગ કરીને, ફક્ત records update થી શકાય છે.

DELETE statement

⊛ table માંથી records delete કરી શકાય છે.

૨૨૦- DELETE FROM Customers
WHERE CustomerName = 'Alfred';

૨૨૦- ફક્ત records delete કરો.

```
DELETE FROM Customers;
```

Warning!

★ DELETE કરી શકાય છે WHERE clause નો ઉપયોગ કરીને, ફક્ત records delete થી શકાય છે.

TOP, LIMIT, FETCH FIRST, and ROWNUM clause

SELECT TOP clause

⊛ return only top records based on sorting order.

⊛ 'Top' SQL servers support, but syntax varies;

1) SQL Server / MS Access

```
SELECT TOP number / Percent column_name(s)
FROM table_name
WHERE condition;
```

2) MySQL

```
SELECT column_name(s)
FROM table_name
WHERE condition
LIMIT number;
```

3) Oracle 12

```
SELECT column_name(s)
FROM table_name
ORDER BY column_name(s)
FETCH FIRST number ROWS ONLY;
```

4) older Oracle

```
SELECT column_name(s)
FROM table_name
WHERE ROWNUM <= number;
```

5) older Oracle (with ORDER BY)

```
SELECT *
FROM (SELECT column_name(s) FROM table_name ORDER BY column_name(s))
WHERE ROWNUM <= number;
```

⊛ Example problem solution.

Print top 3 records;

```
SELECT TOP 3 * FROM Customers;
```

MySQL query ⇒

```
SELECT * FROM Customers
LIMIT 3;
```

Oracle query ⇒

```
SELECT * FROM Customers
FETCH FIRST 3 ROWS ONLY;
```

④ કચોડાએ લેખ વાંચીને લેખ પુનઃલેખન કરી લેખ ૨, બદલી દાખા.

ટીપ્સ: SELECT TOP 50 PERCENT * FROM Customers;

Oracle ક્વેરી ⇒ SELECT * FROM Customers;

FETCH FIRST 50 PERCENT ROWS ONLY;

④ જો ક્વેરી WHERE clause ૨, બદલી દાખા.

ટીપ્સ: Country એ 'Germany' એ ૩ records માં select કરો.

SELECT TOP 3 * FROM Customers

WHERE Country = 'Germany';

MySQL ક્વેરી ⇒ SELECT * FROM Customers
WHERE Country = 'Germany'
LIMIT 3;

Oracle ક્વેરી ⇒ SELECT * FROM Customers
WHERE Country = 'Germany'
FETCH FIRST 3 ROWS ONLY;

MIN() and MAX() functions

④ MIN() function → column એના પૈકી સૌથી નીચા value એ select કરે.

④ MAX() function → column એના પૈકી સૌથી ઊંચા value એ select કરે.

ટીપ્સ: MIN() example;

SELECT MIN(Price) AS SmallestPrice
FROM Products;

MAX() example;

SELECT MAX(Price) AS LargestPrice
FROM Products;

COUNT(), AVG(), SUM() functions

COUNT()

④ કોલમ નો કોલમનું જુદા rows કેટલાં હશે તે જણાવે.

ટીપ્સ: SELECT COUNT(ProductID)
FROM Products;

* NULL values count
જણાવે.

AVG()

④ column એના average એ જણાવે.

ટીપ્સ: SELECT AVG(Price)
FROM Products;

* NULL values ધ્યાનમાં લે.

SUM ()

- * column එකක 'මුළු එකතුව' දක්වයි.

උදා-
SELECT SUM (Quantity)
FROM OrderDetails;

* NULL values හැඳරයි.

LIKE operator

- * column එකක 'මුළු' වූ විශේෂ pattern එකක් සොයා ගැනීමට යොදා ගනී.

- * බොහෝ විට '%' සහ '_' යන wildcards use වෙයි.

% → represents zero, one or multiple characters

_ → represents single character

MS Access වල දී → '%' වෙනුවට '*' ද, '_' වෙනුවට '?' ද, නැතිනම් ටේ.

- * wildcards සෙවීම සඳහා භාවිත කරයි;

WHERE CustomerName LIKE 'a%' ⇒ find any values start with 'a'

WHERE CustomerName LIKE '%a' ⇒ find any values end with 'a'

WHERE CustomerName LIKE '%or%' ⇒ find any values that have 'or' in any position

WHERE CustomerName LIKE '_r%' ⇒ find any values that have 'r' in the second position

WHERE CustomerName LIKE 'a_%' ⇒ find any values that starts with 'a' and are at least 2 characters in length

WHERE CustomerName LIKE 'a__%' ⇒ find any values that starts with 'a' and are at least 3 characters in length

WHERE CustomerName LIKE 'a%o' ⇒ finds any values that starts with 'a' and ends with 'o'

උදා- SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';

} SELECT * FROM Customers
WHERE CustomerName NOT LIKE 'a%';

Wildcards

① String value યોગ્ય રૂઠાં પેટર્ન યોગ્ય પ્રકારે લેવા સરળ.

Wildcard characters in MS Access

- * → represents zero or more characters (જેમ- b* એટલે b, black, blob)
- ? → represents a single character (જેમ- h?t એટલે hot, hat, hit...)
- [] → represents any single character within the brackets (જેમ- h[oa]t એટલે hot, hat)
- ! → represents any character not in the brackets (જેમ- h[!oa]t એટલે hit, het)
- → represents a range of characters (જેમ- c[a-b]t એટલે cat, cbt)
- # → represents any single numeric character (જેમ- 2#5 એટલે 205, 215, 225...)

Wildcard characters in SQL Server

- % → represents zero or more characters
- (underscore) → represents a single character
- [] → represents any single character within brackets
- → represents a range of characters
- ^ → represents any character not in the brackets

} એક સરળ
સમજાવણી
પ્રદાન
થાય.

Aliases

① table યોગ્ય સેબ table column યોગ્ય નામ આપવા માટે (temporary name) દેવા સરળ.

② Alias for columns;

જેમ- `SELECT CustomerID AS ID, CustomerName AS Customer
FROM Customers;`

જેમ- alias name યોગ્ય spaces નીચે આપે; double quotation marks સેબ square brackets સરળ.

`SELECT CustomerName AS Customer, ContactName AS [Contact Person]
FROM Customers;`

જેમ- columns નીચે combined આપે alias name યોગ્ય એક દેવ થાય.

`SELECT CustomerName, Address + ',' + PostalCode + ',' + City
+ ',' + Country AS Address
FROM Customers;`

MySQL query $\Rightarrow$ `SELECT CustomerName,
CONCAT (Address, ',', PostalCode, ',', City, ',', Country) AS Address
FROM Customers;`

④ Alias for tables;

Ex: The statement below CustomerID = 4 in Customer table is linked to orders
select from table; 'Customers' and 'Orders' tables alias "C" and "O"
Aliases names use below.

```
SELECT O.OrderID, O.OrderDate, C.CustomerName  
FROM Customers AS C, Orders AS O  
WHERE C.CustomerName = 'Around the Horn' AND C.CustomerID = O.  
CustomerID;
```

JOIN statement

★ When we join two different tables then we can (combine) columns which are in both tables
which are in both tables.

★ Demo table below Customers and Orders are tables which have CustomerID
column in both tables.

Ex: Below is inner join type which is used.

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate  
FROM Orders
```

```
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

★ JOIN statements are of three types explained below.

★ (INNER) JOIN $\rightarrow$ Returns records that have matching values in both tables

★ LEFT ^(OUTER) ~~(INNER)~~ JOIN $\rightarrow$ Returns all records from the left table and matching
values (records) from the right table

★ RIGHT (OUTER) JOIN $\rightarrow$ Returns all records from the right table and matching
records from the left table

★ FULL (OUTER) JOIN $\rightarrow$ Returns all ~~left~~ records when there is a match
in either left or right table

UNION operator

★ SELECT statements which result sets combined which are in both tables.

★ When we use UNION in SELECT statement then we can combine columns from both
tables.

★ Both columns are of same data types should be used.

★ The SELECT statement then columns should be in same order then we can combine columns.

ຂໍ້ຕົວຢ່າງ: SELECT ~~FROM~~ City FROM Customers
 UNION
 SELECT City FROM Suppliers
 ORDER BY City;

(*) UNION statement ທີ່ນີ້ ຈະບໍ່ເອົາ duplicate values ຕົວຢ່າງ select ຈະບໍ່ເອົາ duplicate values ຕົວຢ່າງ.

UNION ALL statement

(*) UNION statement ທີ່ນີ້ ຈະເອົາ duplicate values ຕົວຢ່າງ. ຕົວຢ່າງ; duplicate values 2, select ຈະບໍ່ເອົາ.

ຂໍ້ຕົວຢ່າງ: SELECT City FROM Customers
 UNION ALL
 SELECT City FROM Suppliers
 ORDER BY City;

ຂໍ້ຕົວຢ່າງ: WHERE ທີ່ນີ້ ຈະເອົາ UNION ຈະບໍ່ເອົາ UNION ALL ຈະບໍ່ເອົາ.

SELECT City, Country FROM Customers
 WHERE Country = 'Germany'
 UNION
 SELECT City, Country FROM Suppliers
 WHERE Country = 'Germany'
 ORDER BY City;

GROUP BY statement

(*) ຈະເອົາ values ທີ່ເກີດຂຶ້ນ rows group ຈະບໍ່ເອົາ.

(*) GROUP BY statement ທີ່ນີ້ ຈະເອົາ aggregate function (COUNT(), MAX(), MIN(), SUM(), AVG()) ຈະບໍ່ເອົາ.

ຂໍ້ຕົວຢ່າງ: SELECT COUNT(CustomerID), Country
 FROM Customers
 GROUP BY Country;

ຂໍ້ຕົວຢ່າງ: ຈະເອົາ ຂໍ້ຕົວຢ່າງ CustomerID ທີ່ນີ້ ຈະບໍ່ເອົາ ຈະບໍ່ເອົາ Customers ຈະບໍ່ເອົາ.

SELECT COUNT(CustomerID), Country
 FROM Customers
 GROUP BY Country
 ORDER BY COUNT(CustomerID) DESC;

HAVING clause

- ★ aggregate functions କ୍ଷେତ୍ର WHERE keyword ଥିବା କ୍ଷେତ୍ର ଉପରେ କାର୍ଯ୍ୟ କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।

```
୧୧୦୮ SELECT COUNT (CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT (CustomerID) > 5;
```

EXISTS Operator

- ★ କୌଣସି record ଥିବା କ୍ଷେତ୍ର test କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।
- ★ EXISTS operator ଥିବା TRUE ହେଉଥିବା ବେଳେ records ଥିବା କ୍ଷେତ୍ର ଉପରେ output ଦିଆଯାଏ।

```
୧୧୦୯ SELECT SupplierName
FROM Suppliers
WHERE EXISTS (SELECT ProductName FROM Products WHERE
Products.SupplierID = Suppliers.SupplierID AND Price < 20);
```

ANY and All operators

- ★ ଏହି operators ଉପରେ column value ଥିବା କ୍ଷେତ୍ର କିମ୍ବା values ଉପରେ କାର୍ଯ୍ୟ କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।

ANY

- ★ Result ଥିବା କ୍ଷେତ୍ର boolean value ଥିବା କ୍ଷେତ୍ର ଉପରେ କାର୍ଯ୍ୟ କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।
- ★ କୌଣସି value ଥିବା କ୍ଷେତ୍ର condition ଥିବା କ୍ଷେତ୍ର କାର୍ଯ୍ୟ କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ; TRUE ହେଉଥିବା ବେଳେ output ଦିଆଯାଏ।

```
୧୧୧୦ SELECT ProductName
FROM Products
WHERE ProductID = ANY
(SELECT ProductID FROM OrderDetails WHERE Quantity = 10);
```

ALL

- ★ Result ଥିବା କ୍ଷେତ୍ର boolean value ଥିବା କ୍ଷେତ୍ର ଉପରେ କାର୍ଯ୍ୟ କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ।
- ★ କୌଣସି values condition ଥିବା କ୍ଷେତ୍ର କାର୍ଯ୍ୟ କରିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ; TRUE ହେଉଥିବା ବେଳେ output ଦିଆଯାଏ।

```
୧୧୧୧ SELECT ALL ProductName
FROM Products
WHERE TRUE;
```


SELECT INTO statement

★ એક table એક table થી data ગ્રહણ table એકાએક copy કરી શકાય છે.

★ કોઈક columns ગ્રહણ table એકાએક copy કરી શકાય syntax;

SELECT *

INTO newtable [IN externaldb]

FROM oldtable

WHERE condition;

★ columns સિદ્ધિ કરી શકાય એક table એકાએક copy કરી શકાય syntax;

SELECT column1, column2, column3,...

INTO newtable [IN externaldb]

FROM oldtable

WHERE condition;

★ ગ્રહણ table એક ઈન્સ્ટોલ થી એક database એકાએક કરી શકાય; IN એક ગ્રહણ table એક database એક પ્રકારે કરી શકાય. ગ્રહણ; એક database એક પ્રકારે કરી શકાય; એક એક કરી શકાય.

Ex:- SELECT * INTO CustomerBackup2017 IN 'Backup.mdb'
FROM Customers;

INSERT INTO SELECT statement

★ એક table એક table થી data copy કરી એક table એકાએક insert કરી શકાય છે.

★ એક table એક table થી data types એકાએક કરી શકાય.

★ કોઈક columns copy કરી શકાય syntax;

INSERT INTO table2

SELECT * FROM table1

WHERE condition;

★ columns સિદ્ધિ કરી શકાય copy કરી શકાય syntax;

INSERT INTO table2 (column1, column2, column3,...)

SELECT column1, column2, column3,...

FROM table1

WHERE condition;

CASE statement

- ⊛ 'if-elseif-else' statements အသုံးပြုမှုကို ခြုံငုံ ဆိုလိုက်ပါသည်။
- ⊛ conditions အချို့မှာ အမှန်၊ မမှန်ဟု ဆုံးဖြတ်နိုင်ပြီး အမှန် ဖြစ်ပါက condition တစ်ခုခုကို အကောင်အထည်ဖော်လုပ်ဆောင်ပြီး၊ အမှန်မဟုတ်ပါက result တစ်ခုခုကို ပြန်လည် စစ်ဆေးခြင်းကို ပြုလုပ်ခြင်း ဖြစ်ပါသည်။
- ⊛ conditions မှန်ပါက အကောင်အထည်ဖော်လုပ်ဆောင်ပြီး၊ ELSE မှန်ပါက အကောင်အထည်ဖော်လုပ်ဆောင်ပြီး၊ NULL ဖြစ်ပါက အကောင်အထည်ဖော်လုပ်ဆောင်ခြင်း ဖြစ်ပါသည်။
- ⊛ Syntax;

CASE

```
WHEN condition1 THEN result1
WHEN condition2 THEN result2
ELSE result
```

END;

- ① 2204 SELECT OrderID, Quantity,

CASE

```
WHEN Quantity > 30 THEN 'The Quantity is greater than 30'
WHEN Quantity = 30 THEN 'The Quantity is 30'
ELSE 'The Quantity is under 30'
```

END AS QuantityText

FROM OrderDetails;

- ② 200 SELECT CustomerName, City, Country
FROM Customers

ORDER BY

(CASE

WHEN City IS NULL THEN Country
ELSE City

END);

NULL functions

- * જ્યારે data tables ે યો યો columns ે data input ી મિતિ પ્રમાણે સ્થાપિત થાય.
- * ુલન પ્રક્રિયા ી; ુલન મૂલ્ય value ુલન add ુલન NULL values ઉમેરવા થાય. ુલન, result ુલન ૨ NULL ુલન.
- * ુલન બંધુક વચ્ચેના બંધોને કમળ કરી ુલન થાય.

MySQL Query $\Rightarrow$ `SELECT ProductName, UnitPrice * (UnitsInStock + IFNULL(UnitsOnOrder, 0))
FROM Products;`

ଉଦାହରଣ, MySQL ଡାଟାବେସ୍ $\Rightarrow$ `SELECT ProductName, UnitPrice * (UnitsInStock + COALESCE(UnitsOnOrder, 0))`
`FROM Products;`

SQL Server ଡାଟାବେସ୍ $\Rightarrow$ `SELECT ProductName, UnitPrice * (UnitsInStock + ISNULL(UnitsOnOrder, 0))`
`FROM Products;`

କେବଳ
`SELECT ProductName, UnitPrice * (UnitsInStock + COALESCE(UnitsOnOrder, 0))`
`FROM Products;`

MS Access ଡାଟାବେସ୍ $\Rightarrow$ `SELECT ProductName, UnitPrice * (UnitsInStock + IIF(IsNull(UnitsOnOrder), 0, UnitsOnOrder))`
`FROM Products;`

Oracle ଡାଟାବେସ୍ $\Rightarrow$ `SELECT ProductName, UnitPrice * (UnitsInStock + NVL(UnitsOnOrder, 0))`
`FROM Products;`

କେବଳ
`SELECT ProductName, UnitPrice * (UnitsInStock + COALESCE(UnitsOnOrder, 0))`
`FROM Products;`

Comments

⊛ Microsoft Access databases ଡାଟାବେସ୍ comments support କରେନାହିଁ।

⊛ Single line comment ଲେଖିବା `--` ଦେଇ ସମ୍ଭାବ୍ୟ ହୁଏ; କିନ୍ତୁ ଏହା ଏକ ଲେଖାଏଁ ଥିବା କିଛି comment ଥିବାକୁ ଦେଖି ସମସ୍ତ ଲେଖିବ।

ଉଦାହରଣ:- `-- This is comment`

⊛ multi line comments ଡାଟାବେସ୍ `/* */` ବ୍ୟବହାର କରେ।

ଉଦାହରଣ:- `/* This
is
multiline
comment */`

operators

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

Arithmetic Operators

20+ SELECT 30+20
 SELECT 30-20
 SELECT 30*20
 SELECT 30/20
 SELECT 30%20

Ⓐ Bitwise operators;

operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR

Ⓐ Comparison Operators;

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or Equal to
<=	Less than or Equal to
<>	Not equal to

20+ SELECT * FROM Products
 WHERE Price = 18;

 WHERE Price > 18;

 WHERE Price < 18;

 WHERE Price >= 18;

 WHERE Price <= 18;

 WHERE Price <> 18;

⊛ Compound operators ;

operator	description
<code>+=</code>	Add equals
<code>-=</code>	Subtract equals
<code>*=</code>	Multiply equals
<code>/=</code>	Divide equals
<code>%=</code>	Modulo equals
<code>&=</code>	Bitwise AND equals
<code>^-=</code>	Bitwise exclusive equals
<code> *=</code>	Bitwise OR equals

⊛ Logical operators ;

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the subquery values meet the condition